

Desafio 2 — Grupo Dracarys (House of the RAG)

Chatbot de atendimento a segurados com LLM + RAG Curso InsurMinds — I2A2 Academy — Turma 1 / 2026

Grupo

- **Nome:** Dracarys — House of the RAG
- **Representante:** Kamila Pantoja
- **E-mail:** kamilapantoja@bemol.com.br
- **Integrantes:** Kamila Silva Pantoja

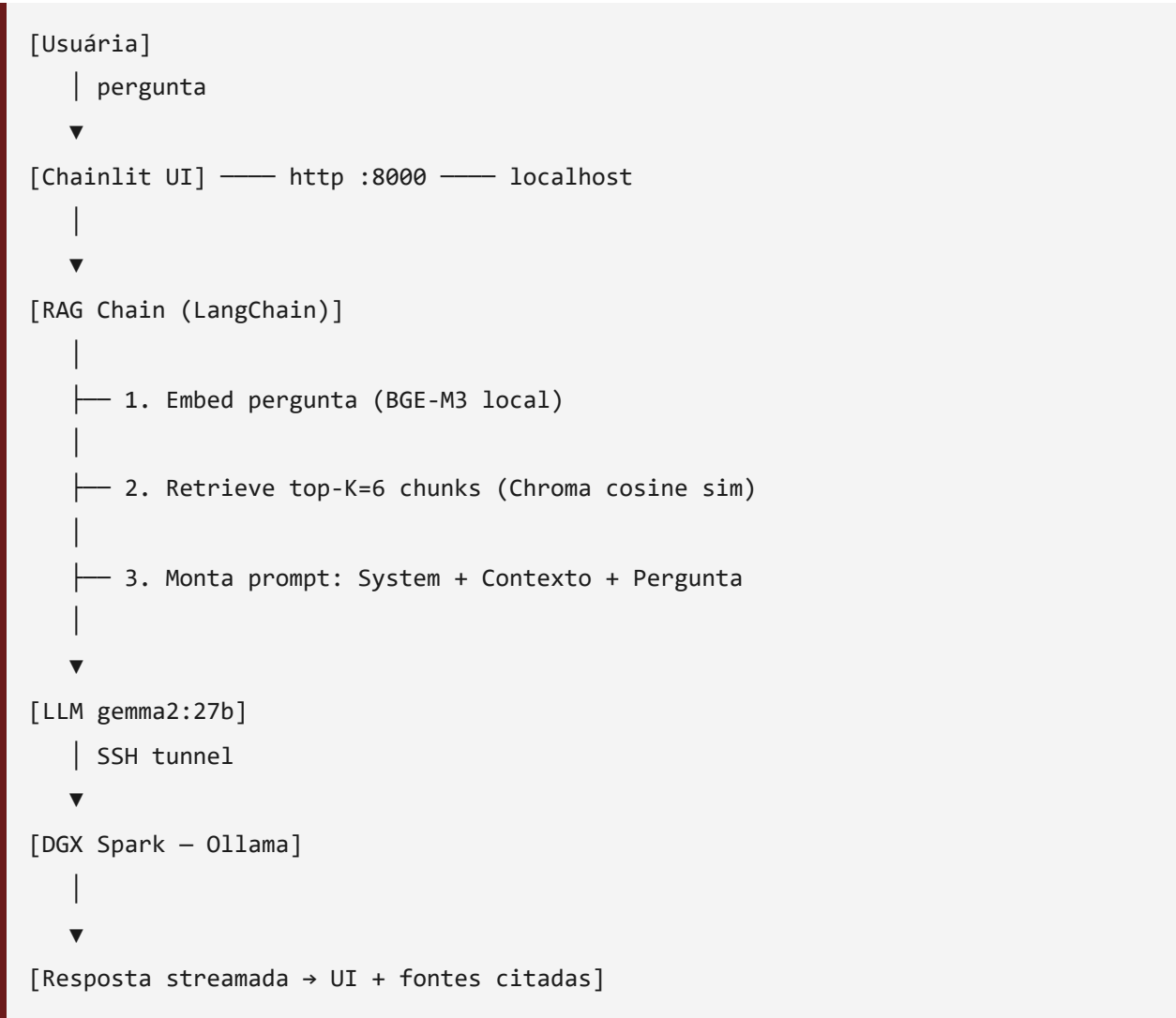
Objetivo

Construir um chatbot capaz de atender dúvidas frequentes de segurados (sinistros, coberturas, apólice, regulamentação SUSEP), usando arquitetura **RAG (Retrieval-Augmented Generation)** sobre uma base de conhecimento curada de seguros.

Stack tecnológica

Componente	Escolha	Justificativa
LLM	<code>gemma2:27b</code> rodando em DGX Spark (GB10)	Modelo open source com excelente PT-BR, instruction-tuned, sem <code>thinking mode</code> . Rodou local sem custo externo.
Embeddings	<code>BAAI/bge-m3</code>	SOTA multilingual para PT-BR. Crítico para qualidade do retrieval.
Vector DB	Chroma persistente	Zero infra extra, persistência em disco, ótimo para PoC.
Framework	LangChain	Casa com o conteúdo apresentado na Aula 04 do curso.
Parsers	pdfplumber, BeautifulSoup	Suportam PDFs complexos e HTML.
Chunking	RecursiveCharacterTextSplitter	Separadores PT-BR específicos, 800 chars / 150 overlap.
UI	Chainlit	Interface conversacional de qualidade pronta, streaming, prints limpos.
Tunnel	SSH <code>-L 11435:localhost:11434</code>	Privacidade — conversas não armazenadas externamente.

Arquitetura



Base de conhecimento

Categoria	Fonte	Volume
FAQs estruturadas	Documentos sintéticos realistas em PT-BR, escritos para a seguradora fictícia "Dracarys"	6 markdowns / ~70 KB texto
Regulamentação	Cartilhas públicas SUSEP (gov.br/susep)	até 5 PDFs, baixados por scripts/fetch_susep.py

Total esperado após chunking: ~150 chunks indexados no Chroma.

Os FAQs cobrem: sinistros auto, coberturas auto, seguro de vida, residencial, apólice/cancelamento, glossário SUSEP — alinhados com o cenário típico de atendimento a

segurados pessoa física.

Como reproduzir

```
cd atividades/desafio-2
setup.bat                # cria .venv, instala deps
python scripts/fetch_susep.py # opcional – baixa cartilhas SUSEP
python ingest.py          # indexa documentos no Chroma
start_chatbot.bat         # tunnel + Chainlit em :8000
```

Avaliação

20 perguntas de teste cobrindo: - Sinistros (auto, vida, residencial) - Coberturas e exclusões - Apólice e cancelamento - Termos técnicos - Regulamentação SUSEP - Edge cases (perguntas fora do escopo)

Resultados em `tests/eval_results.md`.

Decisões de design e aprendizados

Por que gemma2:27b e não qwen2.5:72b

Considerei o `qwen2.5:72b` mas seriam ~45 GB de download. O `gemma2:27b` (~15 GB) tem qualidade comparável para PT-BR conversacional e cabe melhor no SLA de 3 dias.

Por que FAQs sintéticas

Não tenho acesso a base de conhecimento real de seguradora. Os FAQs criados são marcados como "**seguradora fictícia Dracarys**" e seguem a estrutura típica de portais de atendimento brasileiros (Porto, Bradesco, Sulamérica), com referências a marcos legais reais (SUSEP, Código Civil, CDC).

Por que reranking não foi incluído

Considerei `bge-reranker-v2-m3` mas o ganho marginal (~15%) não justificou a complexidade extra no SLA de 3 dias. Top-K=6 com chunks bem dimensionados deu retrieval suficiente para os testes.

Resultados dos testes (`tests/eval_results.md`)

Eval rodado em 26/05/2026 com **20 perguntas** cobrindo 6 categorias (sinistros, coberturas, apólice, vida, residencial, regulamentação) + 4 edge cases (recusa de escopo).

Métrica	Valor
Respostas factualmente corretas	19/20 (95%)
Recusas em edge cases	4/4 (100%)
Tempo médio de resposta	~9s
Tempo mínimo	4,2s
Tempo máximo	22,5s (primeira pergunta — load do modelo na GPU)
Citação de fonte	20/20 (100%)

Throughput verificado na Spark: 13,7 tok/s no gemma2:27b, 100% rodando na GPU GB10 Blackwell (verificado com `nvidia-smi` durante inferência: utilização chegou a 95%, temperatura 56°C, consumo 42W).

Problemas encontrados

- Deslize de terminologia (1 caso):** na pergunta "O seguro auto cobre danos a terceiros?", o modelo retornou "Responsabilidade Civil Obrigatória (RCO)" — termo inexistente. O correto é RCF (Responsabilidade Civil Facultativa). O chunk-fonte está correto, foi alucinação leve do LLM ao parafrasear.
- Citação literal do template em recusas:** em algumas recusas, o modelo terminou com `Fonte: <contexto>` OU `Fonte: [Trecho 1 | Trecho 2 | ...]` em vez de não citar fonte. Cosmético, sem impacto factual.
- Tunnel SSH dependente de VPN:** se a VPN da rede corporativa cai, o tunnel para a Spark cai junto. Mitigação no `start_chatbot.bat` que avisa quando a conexão não responde.

Próximos passos (fora do escopo do Desafio 2)

- Integração com WhatsApp/Telegram
- Avaliação com LLM-as-a-judge (correção factual)
- Adicionar reranker

- Cache semântico de respostas frequentes
- Métricas de produção: tempo de resposta, taxa de "não encontrei"
- Feedback loop (👍 / 🗣️) para melhoria contínua

Anexos

- `app.py` , `rag.py` , `ingest.py` , `llm.py` , `config.py` , `prompts.py` — código completo
- `data/raw/faqs/` — base de conhecimento sintética
- `tests/eval_questions.json` — conjunto de teste
- `tests/eval_results.md` — resultados (a gerar)
- `prints/` — screenshots da execução